

Academy 2026

Piano di studio — Aprile 2026

Esercizi pomeridiani

2 esercizi per ogni giornata di corso

Large Systems s.a.s. — largesyst.it

Giorno 1 — Git e Docker

Esercizio 1 — Il tuo primo repository Git

In questo esercizio metterai in pratica i fondamentali di Git: creare un repository, lavorare con i commit e gestire i branch. Segui i passi nell'ordine indicato e annota il risultato di ogni comando.

Cosa fare:

1. Crea una nuova cartella sul tuo computer chiamata progetto-academy e inizializza al suo interno un repository Git.
2. Crea un file README.md con il seguente contenuto: 'Primo progetto Git - Academy 2026'. Aggiungi il file all'area di staging e crea il tuo primo commit con un messaggio descrittivo.
3. Crea un secondo file chiamato note.txt e scrivi al suo interno almeno tre righe di testo libero (possono essere appunti sulla giornata). Aggiungi il file e crea un nuovo commit.
4. Verifica la storia dei commit con il comando apposito. Quanti commit ha il tuo repository?
5. Crea un nuovo branch chiamato feature/saluto. Spostati su quel branch e crea un file saluto.txt con scritto 'Ciao dal branch feature/saluto!'. Fai un commit.
6. Torna sul branch principale (main o master) e fai il merge del branch feature/saluto. Verifica che il file saluto.txt sia ora presente.
7. BONUS: Prova a modificare lo stesso file su due branch diversi e a risolvere il conflitto di merge che si genera.

▶ **Consegna:** Al termine, esegui `git log --oneline` e copia l'output nel file README.md come sezione 'Storia del progetto'. Fai un commit finale.

Esercizio 2 — Docker: avviare PostgreSQL con un volume

In questo esercizio prenderai confidenza con Docker avviando un database PostgreSQL in un container, configurando le credenziali iniziali, esponendo la porta del servizio e montando un volume per rendere persistenti i dati.

Cosa fare:

8. Verifica che Docker sia correttamente installato eseguendo il comando per visualizzare la versione. Annota la versione di Docker Engine.
9. Scarica (pull) l'immagine ufficiale di PostgreSQL dal Docker Hub. Poi elenca le immagini presenti localmente.
10. Avvia un container basato sull'immagine PostgreSQL in modalità detached (-d), assegnando il nome mio-postgres. Configura le variabili d'ambiente per creare il database academy_db, l'utente academy_user e una password a tua scelta.
11. Mappa la porta 5432 del container sulla porta 5432 del tuo computer e monta un volume Docker per salvare in modo persistente i dati del database.
12. Verifica che il container sia in esecuzione elencando i container attivi. Controlla anche i log del container mio-postgres per assicurarti che PostgreSQL sia partito correttamente.
13. Accedi a PostgreSQL tramite Dbeaver e crea un database per verificare che il cluster sia raggiungibile con l'utente configurato.
14. Ferma il container e riavvialo. Verifica che i dati restino disponibili grazie al volume. Infine elenca i volumi Docker presenti sul sistema.

▶ **Consegna:** Scrivi in un file chiamato docker-postgres-notes.txt i comandi utilizzati per creare il volume, avviare il container, verificarne il funzionamento e testare la persistenza dei dati dopo il riavvio.

Giorno 2 — SQL: basi e interrogazioni

Esercizio 1 — Interrogare un database di una libreria

Utilizza il database MySQL che hai visto installato in Docker nella lezione precedente. Crea la seguente tabella e popolala con i dati forniti, poi esegui le query richieste.

Cosa fare:

15. Crea una tabella LIBRI con le colonne: id (intero), titolo (testo), autore (testo), genere (testo), anno (intero), prezzo (decimale), disponibile (booleano o intero 0/1).
16. Inserisci almeno 10 libri con dati variati (generi diversi, anni tra 1990 e 2024, prezzi tra 8 e 35 euro, alcuni non disponibili). Sii creativo con i titoli!
17. Scrivi una query che recuperi TUTTI i libri, ordinati per prezzo in modo crescente.
18. Scrivi una query che recuperi solo i libri disponibili (disponibile = 1) il cui prezzo è compreso tra 10 e 20 euro.
19. Scrivi una query che recuperi i libri il cui autore contiene la lettera 'a' (usa LIKE). Visualizza solo il titolo e l'autore, con un alias leggibile per le colonne.
20. Scrivi una query che recuperi i libri pubblicati tra il 2000 e il 2015, ordinati per anno decrescente, mostrando solo i primi 5 risultati (usa LIMIT).
21. Scrivi una query che recuperi i libri del genere 'Fantasy' OPPURE del genere 'Fantascienza' usando l'operatore IN.

Consegna: Raccogli tutte le query in un file testo chiamato query-libreria.sql con un commento sopra ogni query che spiega cosa fa. Incolla in un altro file di testo tutto il DDL.

Esercizio 2 — Funzioni e aggregazioni

Utilizzando la stessa tabella LIBRI dell'esercizio precedente, scrivi le seguenti query che fanno uso di funzioni aggregate e funzioni di testo/numeriche.

Cosa fare:

22. Scrivi una query che conti il numero totale di libri presenti nella tabella.
23. Scrivi una query che calcoli il prezzo medio, il prezzo massimo e il prezzo minimo di tutti i libri.
24. Scrivi una query che mostri, per ogni genere, il numero di libri e il prezzo medio. Ordina per numero di libri in modo decrescente. (Usa GROUP BY.)
25. Scrivi una query che usi GROUP BY e HAVING per mostrare solo i generi che hanno più di 2 libri.
26. Scrivi una query che mostri il titolo di ogni libro tutto in maiuscolo (usa una funzione di testo) e la lunghezza del titolo in caratteri.
27. Scrivi una query che calcoli, per ogni libro, il prezzo con IVA al 22% (prezzo * 1.22) arrotondato a due decimali. Mostra titolo, prezzo originale e prezzo con IVA.
28. BONUS: Scrivi una query che usi una funzione di data per visualizzare l'anno corrente e calcoli quanti anni fa è stato pubblicato ogni libro.

Consegna: Aggiungi queste query al file query-libreria.sql. Alla fine del file aggiungi un commento con il risultato numerico della query al punto 1 (quanti libri hai nella tabella).

Giorno 3 — MySQL operativo e introduzione al web Java

Esercizio 1 — Gestire un database di una palestra

In questo esercizio utilizzerai MySQL Server installato localmente per creare e gestire un database completo, includendo operazioni DDL (struttura) e DML (dati), fino alle JOIN tra più tabelle.

Cosa fare:

29. Crea un database chiamato palestra_db. Selezionalo come database attivo.
30. Crea la tabella SOCI con le colonne: id INT AUTO_INCREMENT PRIMARY KEY, nome VARCHAR(50) NOT NULL, cognome VARCHAR(50) NOT NULL, email VARCHAR(100), data_iscrizione DATE, attivo INT DEFAULT 1.
31. Crea la tabella CORSI con le colonne: id INT AUTO_INCREMENT PRIMARY KEY, nome VARCHAR(100) NOT NULL, istruttore VARCHAR(100), max_partecipanti INT, livello VARCHAR(20).
32. Crea la tabella ISCRIZIONI con le colonne: id INT AUTO_INCREMENT PRIMARY KEY, socio_id INT, corso_id INT, data_iscrizione DATE. (Questa tabella collega soci e corsi.)
33. Inserisci almeno 5 soci e 3 corsi. Poi inserisci alcune iscrizioni (ogni socio può essere iscritto a più corsi).
34. Aggiorna l'email di uno dei soci con UPDATE. Poi imposta attivo = 0 per un socio (simulando una disiscrizione). Elimina un'iscrizione.
35. Aggiungi una colonna costo_mensile DECIMAL(6,2) alla tabella CORSI usando ALTER TABLE. Assegna un valore a tutti i corsi.
36. Scrivi una JOIN tra SOCI, ISCRIZIONI e CORSI per ottenere un report che mostri: nome e cognome del socio, nome del corso, data di iscrizione. Filtra solo i soci attivi.

Consegna: Salva tutte le istruzioni SQL in un file palestra.sql. Esegui infine la query GROUP BY per sapere quante iscrizioni ha ciascun corso. Scrvi il DDL in un file separato.

Esercizio 2 — Domande sull'architettura JEE

Rispondi alle seguenti domande in forma scritta. Non è necessario scrivere codice: l'obiettivo è consolidare i concetti teorici con parole tue.

Cosa fare:

37. Cos'è un Web Container? Qual è il suo ruolo in un'applicazione JEE? Fai un esempio di Web Container che conosci.
38. Qual è la differenza tra un sito web statico e un'applicazione web dinamica? Fai un esempio concreto di ciascuno.
39. Cos'è un file WAR? Cosa contiene e perché è importante nel deploy di un'applicazione Java web?
40. Cos'è il protocollo HTTP? Descrivi brevemente cosa succede quando un browser fa una richiesta a un server web (ciclo request-response).
41. A cosa serve il file web.xml in un'applicazione JEE? Fai almeno un esempio di configurazione che può contenere.
42. In base a quanto studiato, qual è la differenza tra JEE e Jakarta EE? Perché è avvenuto questo cambiamento di nome?

Consegna: Scrivi le risposte in un documento di testo o word chiamato domande-jee.txt.

Giorno 4 — Servlet: introduzione, ciclo di vita e scope

Esercizio 1 — La tua prima Servlet con ciclo di vita

Crea un nuovo progetto Web Java (Dynamic Web Project su Eclipse e implementa una Servlet che dimostri il ciclo di vita completo e gestisca una richiesta HTTP di base.

Cosa fare:

43. Crea una Servlet chiamata `BenvenutoServlet` mappata sull'URL `/benvenuto`. Deve rispondere a richieste GET con una pagina HTML che mostri: 'Ciao! Questa è la mia prima Servlet.'
44. Aggiungi alla Servlet l'override dei metodi `init()` e `destroy()`. In ognuno inserisci una `System.out.println` con un messaggio che indica quando il metodo viene chiamato. Verifica sulla console del server quando questi messaggi appaiono.
45. Modifica la Servlet in modo che conti quante volte è stata chiamata (usa una variabile di istanza contatore di tipo `int`). Mostra questo contatore nella risposta HTML.
46. Aggiungi alla Servlet la gestione dei parametri: la URL `/benvenuto?nome=Mario` deve rispondere con 'Ciao, Mario!'. Se il parametro `nome` non è presente, mostra 'Ciao, ospite!'.
47. Crea una seconda Servlet chiamata `InfoServlet` mappata su `/info`. Deve mostrare nella risposta: il metodo HTTP usato, l'URL completa della richiesta, tutti i parametri ricevuti.
48. Configura entrambe le Servlet nel file `web.xml` e verifica che entrambe rispondano correttamente nel browser.

Consegna: Il progetto deve compilare e avviarsi senza errori. Fai uno screenshot del browser con la risposta di `BenvenutoServlet` e uno di `InfoServlet`. Pusha su GitHub il codice completo.

Esercizio 2 — Gestire i 3 scope

In questo esercizio creerai Servlet che usano i tre scope principali di JEE (request, session, application) per capire la differenza tra dati che vivono per una sola richiesta, per tutta la sessione utente, o per tutta la vita dell'applicazione.

Cosa fare:

49. Crea una Servlet `ContatorePaginaServlet` su `/pagina`. Ogni volta che viene chiamata, incrementa un contatore salvato nello scope di APPLICATION (`ServletContext`). Mostra a schermo 'Questa pagina è stata visitata N volte in totale (da tutti gli utenti).'
50. Crea una Servlet `ContatoreUtenteServlet` su `/utente`. Usa lo scope di SESSION per contare quante volte il singolo utente ha visitato la pagina. Mostra 'Hai visitato questa pagina N volte.'
51. Crea una Servlet `EchoServlet` su `/echo`. Prende un parametro testo dalla richiesta, lo salva nello scope di REQUEST, e fa un forward a una JSP (o risponde direttamente) mostrando il testo ricevuto. Questo scope si azzera ad ogni nuova richiesta.
52. Testa il comportamento dei tre scope: apri due finestre del browser diverse (o una normale e una in incognito) e visita le pagine. Osserva le differenze nel contatore: cosa cambia e cosa rimane uguale?
53. Scrivi una breve spiegazione (anche in un commento nel codice) di quando useresti ciascuno dei tre scope in un'applicazione reale. Fai un esempio concreto per ognuno.

Consegna: Il progetto deve funzionare con tutte e tre le Servlet. Fai uno screenshot del browser con la risposta di `BenvenutoServlet` e uno di `InfoServlet`. Pusha su GitHub il codice completo.

Giorno 5 — Servlet avanzate: filtri, listener e sessioni

Esercizio 1 — Un filtro di logging e redirect

In questo esercizio implementerai un Filtro che intercetta tutte le richieste in arrivo e le registra su console, e gestirai il redirect HTTP per simulare una funzionalità di 'pagina non disponibile'.

Cosa fare:

54. Crea un filtro chiamato `LoggingFilter` che intercetti TUTTE le richieste all'applicazione (pattern `/*`). Per ogni richiesta, stampa su console: timestamp, metodo HTTP, URL richiesta.
55. Modifica il `LoggingFilter` per misurare anche il tempo di esecuzione della richiesta: salva il tempo prima di chiamare `chain.doFilter()`, poi calcola il tempo trascorso dopo e stampalo su console.
56. Crea una Servlet `MaintenanceServlet` su `/manutenzione` che mostra una pagina HTML con scritto 'Sito in manutenzione, torna presto!'.
57. Crea un secondo filtro `MaintenanceFilter` che, se una variabile booleana `IN_MANUTENZIONE` è impostata a `true`, reindirige tutte le richieste (tranne quella a `/manutenzione` stessa) verso `/manutenzione`. Imposta la variabile a `true` e verifica il comportamento.
58. Implementa in una Servlet la differenza pratica tra `forward` e `redirect`: crea `/azione-forward` e `/azione-redirect`. Entrambe devono portare l'utente su una pagina di 'destinazione', ma una con `forward` e l'altra con `sendRedirect`. Apri i DevTools del browser e osserva la differenza nel numero di richieste HTTP.

Consegna: Documenta nel codice con commenti la differenza tra `forward` e `redirect`, e spiega quando useresti l'uno o l'altro. Pusha su GitHub il codice completo.

Esercizio 2 — Sistema di login con sessione

Costruisci un mini sistema di login basato sulle sessioni HTTP. Non è necessario un database reale: usa credenziali hard-coded. L'obiettivo è capire come le sessioni tracciano lo stato dell'utente.

Cosa fare:

59. Crea un form HTML (in un file `.html` o in una Servlet) con due campi: username e password, e un pulsante 'Accedi'. Il form deve inviare i dati in POST alla Servlet `LoginServlet` su `/login`.
60. Crea `LoginServlet` che gestisce la POST: controlla se username è 'admin' e password è '1234'. Se corretto, salva l'username nella sessione con `setAttribute` e fai redirect a `/dashboard`. Se sbagliato, reindirizza a `/login?errore=1`.
61. Crea `DashboardServlet` su `/dashboard`. All'inizio del metodo `doGet()`, controlla se l'attributo username è presente nella sessione. Se non c'è, fai redirect a `/login` (utente non autenticato). Se c'è, mostra 'Benvenuto, [username]! Sei nella dashboard.'
62. Aggiungi alla dashboard un link o un bottone 'Logout'. Crea `LogoutServlet` su `/logout` che invalida la sessione (`session.invalidate()`) e reindirizzi a `/login`.
63. Aggiungi al form di login la gestione del parametro errore: se l'URL contiene `?errore=1`, mostra un messaggio rosso 'Credenziali non valide. Riprova.'
64. Crea un `SessionListener` (implementa `HttpSessionListener`) che stampa su console ogni volta che viene creata o distrutta una sessione, con il relativo ID di sessione.

Consegna: Testa l'intero flusso: login errato, messaggio errore, login corretto, dashboard, logout. Verifica che dashboard non sia più accessibile. Pusha su GitHub il codice completo.

Giorno 7 — JSP, JavaBeans e JSTL

Esercizio 1 — Pagine dinamiche con JSP e Scriptlet

In questo esercizio utilizzerai JSP per creare pagine web dinamiche. Dovrai usare scriptlet, direttive ed expression language per generare contenuto HTML in modo dinamico.

Cosa fare:

65. Crea una pagina JSP chiamata `benvenuto.jsp` che mostri il giorno e l'ora correnti usando Java all'interno di uno scriptlet (`<% ... %>`). Usa la classe `java.util.Date` o `java.time.LocalDateTime`.
66. Crea una pagina `lista-numeri.jsp` che, usando un ciclo `for` dentro uno scriptlet, generi una lista HTML (`<ul><li>`) con i numeri da 1 a 10.
67. Aggiungi alla `lista-numeri.jsp` la direttiva `page` per impostare il `content-type` a `text/html` e il `charset` a `UTF-8`. Poi includi un'altra JSP (`header.jsp`) con la direttiva `include`, contenente un semplice titolo della pagina.
68. Crea una pagina `calcolatrice.jsp` che legga due parametri dalla URL (`num1` e `num2`), li sommi e mostri il risultato. Esempio: `calcolatrice.jsp?num1=5&num2=3` deve mostrare `'5 + 3 = 8'`. Gestisci il caso in cui i parametri non siano presenti (mostra un messaggio di istruzione).
69. Modifica `calcolatrice.jsp` usando JSP Actions: usa `<jsp:include>` per includere un `footer.jsp`. Poi crea una pagina `redirect-test.jsp` che usi `<jsp:forward>` per reindirizzare a `benvenuto.jsp`.

 **Consegna:** Tutte le JSP devono funzionare nel browser senza errori. Aggiungi un commento JSP (`<%-- commento --%>`) in ogni file spiegando cosa fa quella pagina.

Esercizio 2 — JavaBeans e JSTL

In questo esercizio utilizzerai il pattern JavaBeans con JSP e poi semplificherai il codice usando i tag JSTL core, eliminando il codice Java dalle pagine.

Cosa fare:

70. Crea un JavaBean chiamato `Prodotto` con le proprietà: `nome` (String), `prezzo` (double), `disponibile` (boolean). Il bean deve avere il costruttore vuoto e tutti i getter/setter.
71. Crea una JSP `prodotto.jsp` che usi `<jsp:useBean>` per creare un'istanza di `Prodotto`, poi `<jsp:setProperty>` per impostare `nome='Laptop'` e `prezzo=999.99`. Infine usa `<jsp:getProperty>` per visualizzare i valori.
72. Crea una Servlet `ProdottiServlet` che crei una `List<Prodotto>` con almeno 5 prodotti (con prezzi e disponibilità variati) e la salvi come attributo di request. Poi fa forward a una JSP `lista-prodotti.jsp`.
73. In `lista-prodotti.jsp`, usa JSTL core (taglib uri="`http://java.sun.com/jsp/jstl/core`") con il tag `<c:forEach>` per iterare sulla lista e visualizzare ogni prodotto in una riga di una tabella HTML.
74. Aggiungi un tag `<c:if>` per mostrare `'Disponibile'` in verde o `'Non disponibile'` in rosso in base alla proprietà `disponibile`. Poi usa `<c:choose>`, `<c:when>`, `<c:otherwise>` per mostrare `'Economico'`, `'Medio'`, `'Costoso'` in base al prezzo (`< 20`, `tra 20 e 100`, `> 100`).
75. Aggiungi nella stessa JSP il totale dei prezzi usando `<c:forEach var>Status>` o calcolandolo nella Servlet. Mostra in fondo alla tabella: `'Totale prodotti: N | Prezzo medio: X €'`.

 **Consegna:** La pagina `lista-prodotti.jsp` deve contenere ZERO scriptlet Java: tutto deve usare EL (`${...}`) e tag JSTL. Questo è il principio della separazione tra logica e presentazione.

Giorno 8 — Spring Boot: fondamenti del progetto

Esercizio 1 — Il tuo primo progetto Spring Boot

Crea da zero un progetto Spring Boot usando Spring Initializr (start.spring.io) e implementa un semplice REST controller. Poi configura DevTools e Actuator.

Cosa fare:

76. Vai su start.spring.io e genera un progetto con: Group=com.academy, Artifact=primo-progetto, dipendenze: Spring Web, Spring Boot DevTools, Spring Boot Actuator. Scarica, decomprimi e aprilo nel tuo IDE.
77. Esplora la struttura del progetto: individua pom.xml, la classe main @SpringBootApplication, e la cartella resources. Apri pom.xml e identifica il parent spring-boot-starter-parent e la dipendenza spring-boot-starter-web.
78. Crea un package com.academy.controller e al suo interno una classe SalutoController annotata con @RestController. Crea un endpoint GET su /saluto che restituisce la stringa 'Ciao dal mio primo Spring Boot!'
79. Avvia l'applicazione ed apri http://localhost:8080/saluto nel browser. Poi verifica che Spring Boot Actuator funzioni aprendo http://localhost:8080/actuator/health.
80. Nel file application.properties, aggiungi la configurazione per: cambiare la porta del server a 8081, impostare management.endpoints.web.exposure.include=* per esporre tutti gli endpoint di Actuator. Riavvia e verifica.
81. Modifica una stringa nel tuo controller (es. aggiungi '!!!' alla risposta) e salva il file. Verifica che DevTools ricarichi automaticamente l'applicazione senza dover riavviare manualmente.

 **Consegna:** L'applicazione deve avviarsi sulla porta 8081, rispondere su /saluto e avere Actuator funzionante. Aggiungi al controller un endpoint /info che restituisce il tuo nome, come JSON: {"autore": "Mario Rossi"}.

Esercizio 2 — Properties personalizzate e configurazione

Impara a iniettare configurazioni personalizzate nell'applicazione Spring Boot usando @Value e application.properties. Poi esplora i profili di configurazione.

Cosa fare:

82. Nel file application.properties aggiungi tre proprietà personalizzate: app.nome=Academy App app.versione=1.0.0 app.messaggio-benvenuto=Benvenuto nella nostra applicazione!
83. Nel SalutoController, inietta queste tre proprietà usando l'annotazione @Value (es. @Value("\${app.nome}") private String appNome;).
84. Crea un endpoint GET /app-info che restituisce una stringa contenente tutte e tre le proprietà: 'App: [nome], Versione: [versione], Messaggio: [messaggio]'.
85. Crea un file application-dev.properties con messaggio diverso (es. 'Sei in ambiente di SVILUPPO!'). Crea anche application-prod.properties con un altro messaggio. Attiva il profilo dev aggiungendo spring.profiles.active=dev in application.properties e verifica che il messaggio cambi.
86. Crea un endpoint GET /configurazione-server che legga e restituisca la porta su cui è in ascolto il server (puoi iniettarla con @Value("\${server.port}")).
87. BONUS: Crea una classe @Component chiamata AppConfig con un metodo che stampi su console, all'avvio dell'applicazione, tutte le proprietà personalizzate. Usa @PostConstruct per eseguirlo dopo l'inizializzazione del bean.

 **Consegna:** Testa tutti gli endpoint. Poi documenta in un file README.md del progetto i passaggi per avviare l'applicazione e la lista degli endpoint disponibili con una breve descrizione.

Giorno 9 — Spring Core e introduzione a JPA/Hibernate

Esercizio 1 — Dependency Injection e gestione dei Bean

In questo esercizio esplorerai il cuore di Spring: Inversion of Control e Dependency Injection. Creerai componenti con diverse modalità di iniezione e esplorerai gli scope dei bean.

Cosa fare:

88. Crea un'interfaccia `SalutoService` con un metodo `String getSaluto()`. Crea due implementazioni: `SalutoItalianoService` (restituisce 'Buongiorno!') e `SalutoIngleseService` (restituisce 'Good morning!'). Annota entrambe con `@Service`.
89. Crea un `@RestController` chiamato `SalutoController` che inietta `SalutoService` tramite Constructor Injection (`@Autowired` sul costruttore). Un endpoint `GET /saluto` deve restituire il saluto del service.
90. Poiché hai due implementazioni, Spring non saprà quale scegliere. Usa `@Primary` su `SalutoItalianoService` per renderlo il default. Verifica che funzioni. Poi prova a usare `@Qualifier` sul controller per iniettare invece `SalutoIngleseService`.
91. Crea un `@Component` chiamato `ContatoreBeanSingleton`. Aggiungi una variabile `int` `contatore` e un metodo `incrementa()`. Non specificare scope: sarà singleton per default. Crea un endpoint che lo usi e chiama `incrementa()` ad ogni richiesta. Cosa osservi?
92. Ripeti il punto 4 con un `@Component` `ContatoreBeanPrototype` annotato con `@Scope("prototype")`. Inietta nel controller. Cosa cambia? Spiega perché con un commento nel codice.
93. Crea un `@Component` `SalutoComplesso` con un metodo `@PostConstruct init()` che stampa 'Bean inizializzato!' e un metodo `@PreDestroy cleanup()` che stampa 'Bean distrutto!'. Avvia e ferma l'applicazione e osserva i messaggi in console.

Consegna: Scrivi un commento in cima al controller che spieghi con parole tue la differenza tra `@Primary` e `@Qualifier`, e la differenza tra scope singleton e prototype.

Esercizio 2 — Primo progetto con JPA/Hibernate

Configura un progetto Spring Boot con JPA e H2 (database in memoria, non richiede installazione) e crea la tua prima entità. Verificheremo che Hibernate generi la tabella.

Cosa fare:

94. Crea un nuovo progetto Spring Boot su start.spring.io con le dipendenze: Spring Web, Spring Data JPA, H2 Database.
95. Nel file `application.properties` configura il datasource H2:
`spring.datasource.url=jdbc:h2:mem:testdb` `spring.datasource.driver-class-name=org.h2.Driver` `spring.jpa.database-platform=org.hibernate.dialect.H2Dialect`
`spring.h2.console.enabled=true` `spring.jpa.show-sql=true`
96. Crea una classe `Studente` nel package `com.academy.entity` con le annotazioni JPA: `@Entity`, `@Table(name="studenti")`. Aggiungi i campi: `id` (`@Id`, `@GeneratedValue`), `nome`, `cognome`, `email`. Genera getter e setter.
97. Avvia l'applicazione. Apri il browser su `http://localhost:8080/h2-console`, connettiti al database `testdb` e verifica che la tabella `STUDENTI` sia stata creata automaticamente da Hibernate.

Consegna: Fai uno screenshot della H2 Console con i dati della tabella `STUDENTI`

Giorno 10 — JPA CRUD e basi REST

Esercizio 1 — CRUD completo con JPA

Estendendo il progetto dell'esercizio precedente (o creandone uno nuovo), implementerai tutte le operazioni CRUD (Create, Read, Update, Delete) usando JPA, sia con EntityManager che con Spring Data JPA.

Cosa fare:

98. Aggiorna l'entità `Studente` aggiungendo: `data_nascita` (`LocalDate`), `corso_laurea` (`String`). Aggiungi anche l'annotazione `@Column(nullable=false)` su nome e cognome. Riavvia: Hibernate deve aggiornare la tabella.
99. Crea uno `StudenteDAO` (o usa direttamente il `Repository`). Implementa un metodo `findAll()` che restituisce tutti gli studenti, e un `findById(int id)` che restituisce uno studente per ID.
100. Implementa un metodo `save(Studente s)` per salvare un nuovo studente, e un metodo `update(Studente s)` per aggiornare uno esistente. Usa `@Transactional` dove necessario.
101. Implementa un metodo `deleteById(int id)` che elimina uno studente per ID.
102. Crea una query JPQL personalizzata: `findByCorsoLaurea(String corso)` che restituisce tutti gli studenti iscritti a un dato corso. Usa `@Query` nel `Repository` oppure scrivi la JPQL con `EntityManager`.
103. Usa il `DataLoader` (`CommandLineRunner`) per testare TUTTE le operazioni CRUD: crea 3 studenti, leggili tutti, aggiorna il corso di laurea del primo, elimina il terzo, poi fai una query per corso. Stampa i risultati su console.
104. Aggiungi la generazione automatica della tabella: metti `spring.jpa.hibernate.ddl-auto=create-drop` e verifica che ad ogni riavvio la tabella venga ricreata.

 **Consegna:** La console dell'applicazione deve mostrare (grazie a `show-sql=true`) tutte le query SQL generate da Hibernate durante il test CRUD. Copia le query in un file `hibernate-queries.txt`.

Esercizio 2 — Il tuo primo REST Controller

Crea i tuoi primi endpoint REST per esporre i dati degli studenti. Introduci il concetto di risposta JSON, metodi HTTP e codici di stato.

Cosa fare:

105. Crea un `@RestController` `StudenteController` con il mapping base `/api/studenti`.
106. Implementa un endpoint `GET /api/studenti` che restituisce la lista di tutti gli studenti (Spring la converte automaticamente in JSON grazie a Jackson). Testalo nel browser o con Postman.
107. Implementa un endpoint `GET /api/studenti/{id}` che restituisce un singolo studente per ID. Se lo studente non esiste, restituisci un `ResponseEntity` con codice 404.
108. Implementa un endpoint `POST /api/studenti` che accetta nel body un oggetto JSON con i dati di un nuovo studente (`@RequestBody`), lo salva e restituisce lo studente salvato con codice 201 (`Created`).
109. Implementa un endpoint `PUT /api/studenti/{id}` che aggiorna i dati di uno studente esistente. Se l'ID non esiste, risponde con 404.
110. Implementa un endpoint `DELETE /api/studenti/{id}` che elimina uno studente. Risponde con 204 (`No Content`) se l'eliminazione ha successo, 404 se non trovato.

111. Testa tutti gli endpoint con Postman (o con curl) e documenta le chiamate nel file README.md del progetto.

 **Consegna:** Tutti e 5 gli endpoint devono funzionare correttamente. Crea una collection Postman (o un file .http se usi VS Code) con tutte le chiamate di test.

Giorno 12 — REST avanzato e introduzione alla sicurezza

Esercizio 1 — Path variables, exception handling e service layer

Raffinerai la tua API REST introducendo il Service Layer per separare la logica di business dal controller, gestirai le eccezioni in modo centralizzato e utilizzerai path variables e query params.

Cosa fare:

- Refactoring: estrai la logica CRUD dal controller e crea un `@Service` `StudenteService`. Il controller deve delegare TUTTE le operazioni al service. Il service usa il repository.
- Crea un'eccezione personalizzata `StudenteNotFoundException` (che estende `RuntimeException`). Lanciala nel service quando un'operazione cerca uno studente con ID non esistente.
- Crea un `@RestControllerAdvice` `GlobalExceptionHandler` con un metodo annotato `@ExceptionHandler(StudenteNotFoundException.class)` che restituisca una risposta JSON strutturata: `{"errore": "Studente non trovato", "id": 99, "timestamp": "..."} con codice HTTP 404.`
- Aggiungi un endpoint `GET /api/studenti/cerca` con un parametro query `@RequestParam` `String cognome` che restituisce tutti gli studenti con quel cognome. Se nessuno trovato, restituisce lista vuota (non 404).
- Aggiungi un endpoint `GET /api/studenti/{id}/corso` che restituisce solo il corso di laurea dello studente con quell'ID, come stringa JSON. Se lo studente non esiste, usa l'eccezione del punto 2.
- Testa la gestione degli errori: chiama `/api/studenti/9999` e verifica che la risposta abbia codice 404 e il corpo JSON corretto con timestamp. Usa Postman per verificare gli header della risposta.

Consegna: L'architettura finale deve rispettare la separazione: Controller -> Service -> Repository.

Esercizio 2 — Prima configurazione della sicurezza REST

Aggiungi Spring Security al progetto e configuralo per proteggere gli endpoint dell'API. Implementa l'autenticazione HTTP Basic e una prima distinzione tra endpoint pubblici e protetti.

Cosa fare:

- Aggiungi al `pom.xml` la dipendenza `spring-boot-starter-security`. Riavvia l'applicazione: cosa succede se provi ad accedere agli endpoint?
- Crea una classe `SecurityConfig` annotata con `@Configuration` e `@EnableWebSecurity`. Crea un bean `SecurityFilterChain` che configuri: `GET /api/studenti` accessibile senza autenticazione, tutti gli altri endpoint richiedono autenticazione.
- Configura un utente in-memory nel `SecurityConfig` con `username='admin'`, `password='admin123'` e ruolo `ADMIN`.
- Testa con Postman: la `GET /api/studenti` deve funzionare senza credenziali. La `POST /api/studenti` deve richiedere Basic Auth con `username/password`. Cosa risponde il server se inserisci credenziali sbagliate? (Nota il codice HTTP)
- Aggiungi un secondo utente con `username='user'` e ruolo `USER`. Configura la security in modo che: gli utenti con ruolo `USER` possano solo fare `GET`, solo gli `ADMIN` possano fare `POST`, `PUT` e `DELETE`.
- NOTA: Disabilita la protezione CSRF per le API REST (`csrf().disable()`) nel `SecurityConfig`

Consegna: Crea una tabella in formato testo (nel `README.md`) che mostri per ogni endpoint la URL, il metodo HTTP, e chi può accedervi (tutti / USER / ADMIN).

Giorno 13 — CRUD REST completo, Spring Data REST e OpenAPI

Esercizio 1 — API REST completa con Spring Data JPA

Completa la tua API REST migrando dal DAO manuale a Spring Data JPA, sfruttando i metodi built-in del repository e le query derivate dal nome del metodo.

Cosa fare:

- Migrazione: sostituisci il tuo DAO con un'interfaccia `StudenteRepository` che estende `JpaRepository<Studente, Integer>`. Verifica che tutte le funzionalità (`findAll`, `findById`, `save`, `deleteById`) continuino a funzionare.
- Aggiungi al repository questi metodi con query derivate: `findByCognome(String cognome)`, `findByCorsoLaurea(String corso)`, `findByNomeContaining(String parte)`.
- Esponi questi nuovi metodi nel controller aggiungendo endpoint: `GET /api/studenti/cognome/{cognome}` e `GET /api/studenti/corso/{corso}`.
- Implementa `PATCH` su `/api/studenti/{id}`: deve aggiornare SOLO i campi presenti nel body JSON (partial update), lasciando inalterati quelli non specificati. Usa `@RequestBody Map<String, Object>` per ricevere solo i campi da aggiornare.
- Aggiungi la paginazione: l'endpoint `GET /api/studenti` deve accettare parametri `page` e `size` (es. `/api/studenti?page=0&size=5`). Usa `Pageable` di Spring Data. La risposta deve includere informazioni sulla pagina corrente, dimensione e totale.
- Aggiungi l'ordinamento: `/api/studenti?sort=cognome,asc` deve restituire gli studenti ordinati per cognome. Verifica che funzioni combinato con la paginazione.

Consegna: Testa paginazione e ordinamento con Postman. Salva i JSON delle risposte (che mostrano i metadati della paginazione) nel file `README.md` del progetto.

Esercizio 2 — Documentazione con OpenAPI/Swagger

Aggiungi la documentazione automatica all'API usando SpringDoc OpenAPI (Swagger). Una buona API deve essere auto-documentata!

Cosa fare:

- Aggiungi al `pom.xml` la dipendenza `springdoc-openapi-starter-webmvc-ui`. Riavvia e apri `http://localhost:8080/swagger-ui.html`. Cosa vedi?
- Arricchisci la documentazione dell'API usando le annotazioni OpenAPI nel controller. Aggiungi `@Tag(name="Studenti", description="Gestione studenti")` alla classe. Aggiungi `@Operation(summary="Descrizione breve")` a ogni endpoint.
- Aggiungi `@ApiResponse` per documentare i possibili codici di risposta di ogni endpoint. Ad esempio, per `GET /api/studenti/{id}`: 200 OK con lo studente, 404 se non trovato.
- Nella classe `Studente`, aggiungi `@Schema(description="...")` su ogni campo per documentare il significato di ogni proprietà. Usa `@Schema(example="Mario")` per fornire esempi.
- Configura le informazioni generali dell'API: crea un bean `OpenAPI` con titolo, versione, descrizione, e informazioni di contatto (il tuo nome e email).
- Apri Swagger UI e testa direttamente un endpoint `GET` e un `POST` dall'interfaccia. Questo è uno strumento molto usato nei team di sviluppo per testare e comunicare le API!

Consegna: Fai uno screenshot della Swagger UI con la tua API documentata. Poi esporta la specifica OpenAPI in formato JSON aprendo `http://localhost:8080/v3/api-docs` e salvala come `api-spec.json`.

Giorno 14 — Sicurezza REST completa e avvio advanced mappings

Esercizio 1 — Sicurezza con JDBC e BCrypt

Migra la configurazione della sicurezza da utenti in-memory a utenti memorizzati nel database, e implementa la cifratura sicura delle password con BCrypt.

Cosa fare:

112. Crea nel database due tabelle per la sicurezza Spring: users (username VARCHAR, password VARCHAR, enabled BOOLEAN) e authorities (username VARCHAR, authority VARCHAR). Popolale con almeno 2 utenti: admin/ROLE_ADMIN e user/ROLE_USER.
113. Modifica SecurityConfig per usare JdbcUserDetailsManager al posto degli utenti in-memory. Configura il datasource. Riavvia e verifica che il login funzioni con gli utenti del database.
114. Installa BCryptPasswordEncoder come bean nel SecurityConfig. Aggiorna le password nel database codificandole con BCrypt (puoi usare un piccolo test main per generare gli hash). Aggiorna il PasswordEncoder nel config.
115. Verifica che il login funzioni ancora con le password BCrypt. Prova a leggere la colonna password nel database: non deve essere più leggibile in chiaro.
116. Aggiungi un endpoint POST /api/auth/registrazione che accetta {username, password, ruolo} e crea un nuovo utente nel database con la password cifrata. Gestisci il caso in cui l'username esista già (risposta 409 Conflict).
117. Crea tabelle personalizzate invece delle tabelle predefinite di Spring (es. utente e ruolo con colonne diverse). Configura Spring Security per usare le tue tabelle con query SQL personalizzate.

 **Consegna:** Dimostra il flusso completo: registra un nuovo utente via API, poi usa le sue credenziali per autenticarti su un endpoint protetto. Documenta le query SQL usate per creare le tabelle.

Esercizio 2 — Prima relazione @OneToOne

Inizia a lavorare con i mapping avanzati JPA creando una relazione One-to-One tra due entità. Modellerai la relazione tra uno Studente e il suo Profilo personale.

Cosa fare:

118. Crea un'entità ProfiloStudente con: id, bio (TEXT), linkedin (String), foto_url (String), data_creazione (LocalDateTime). Annota con @Entity.
119. Aggiungi all'entità Studente un campo ProfiloStudente profilo. Configuralo con @OneToOne e @JoinColumn(name="profilo_id") per creare la foreign key.
120. Aggiungi alla relazione cascade=CascadeType.ALL: questo farà sì che quando salvi uno Studente con un Profilo associato, anche il Profilo venga salvato automaticamente.
121. Nel DataLoader, crea almeno 2 studenti con profilo e 1 senza. Verifica nella H2 Console che la foreign key sia presente nella tabella STUDENTI.
122. Crea un endpoint GET /api/studenti/{id}/profilo che restituisce il profilo dello studente (o 404 se non ha un profilo).

123. Implementa la relazione bidirezionale: aggiungi in `ProfiloStudente` un campo `@OneToOne(mappedBy="profilo") Studente studente`. Fai attenzione alla serializzazione JSON circolare: usa `@JsonIgnore` su uno dei lati.
124. Testa la cancellazione: elimina uno studente con il profilo associato e verifica (grazie a `CascadeType.ALL`) che anche il profilo venga eliminato.

 **Consegna:** Diagramma ER (anche disegnato a mano) che mostri le tabelle `STUDENTI` e `PROFILO_STUDENTE` con la foreign key. Carica il diagramma come immagine nel progetto.

Giorno 15 — Advanced mappings: One-to-One e One-to-Many

Esercizio 1 — Relazione bidirezionale One-to-Many

Modella una relazione realistica One-to-Many: un corso di laurea ha molti studenti. Lavorerai con *fetch types*, *lazy loading* e *JOIN FETCH* per ottimizzare le query.

Cosa fare:

125. Crea un'entità `CorsoLaurea` con: `id`, `nome` (String, unico), `dipartimento` (String), `durata_anni` (int). Annota con `@Entity`.
126. Aggiungi in `CorsoLaurea` una lista `@OneToMany(mappedBy="corsoLaurea", cascade=CascadeType.ALL) List<Studente> studenti`. Aggiungi in `Studente` `@ManyToOne @JoinColumn(name="corso_laurea_id") CorsoLaurea corsoLaurea`.
127. Nel `DataLoader`, crea 3 corsi di laurea e assegna ogni studente a un corso. Verifica nella H2 Console che la colonna `CORSO_LAUREA_ID` sia presente in `STUDENTI`.
128. Crea un endpoint `GET /api/corsi` che restituisce la lista di tutti i corsi di laurea. Verifica cosa succede con la serializzazione JSON: potresti incorrere in riferimenti circolari. Usa `@JsonManagedReference` / `@JsonBackReference` o `@JsonIgnore` per risolverli.
129. Sperimenta con il `fetch type`: aggiungi `FetchType.EAGER` alla relazione in `CorsoLaurea` e osserva le query SQL in console (`show-sql=true`). Poi cambia a `FetchType.LAZY` e confronta.
130. Crea un endpoint `GET /api/corsi/{id}/studenti` che restituisce tutti gli studenti di un corso. Con `LAZY` loading, questo potrebbe generare il problema N+1. Risolvi usando `@Query` con `JOIN FETCH` nel repository.
131. Aggiungi in `CorsoLaurea` un metodo che conti gli studenti iscritti. Crea una query JPQL che restituisca il numero di studenti per ogni corso: `SELECT c.nome, COUNT(s) FROM CorsoLaurea c JOIN c.studenti s GROUP BY c.nome`.

 **Consegna:** Lancia l'applicazione con `show-sql=true` e confronta il numero di query generate con `LAZY` vs `JOIN FETCH`. Copia le due versioni di SQL in un file `confronto-query.txt` con un commento esplicativo.

Esercizio 2 — Esami e relazione unidirezionale

Modella la relazione tra uno `Studente` e i suoi `Esami`: uno studente può sostenere molti esami. Questa è una relazione One-to-Many unidirezionale (solo `Studente` conosce i suoi `Esami`).

Cosa fare:

132. Crea un'entità `Esame` con: `id`, `materia` (String), `voto` (int, tra 18 e 30), `data_esame` (LocalDate), `lode` (boolean).
133. In `Studente` aggiungi `@OneToMany(cascade=CascadeType.ALL) @JoinColumn(name="studente_id") List<Esame> esami`. Nota: non c'è `mappedBy` perché è unidirezionale. Verifica che la tabella `ESAMI` abbia la colonna `STUDENTE_ID`.
134. Nel `DataLoader`, aggiungi almeno 3 esami per il primo studente e 2 per il secondo. Un esame con `lode` deve avere `voto=30`.
135. Crea un endpoint `GET /api/studenti/{id}/esami` che restituisce la lista degli esami dello studente, ordinati per data.
136. Crea un endpoint `POST /api/studenti/{id}/esami` che aggiunge un nuovo esame a uno studente esistente. Valida che il voto sia tra 18 e 30 (usa `@Valid` e `@Min/@Max` sull'entità, dopo aver aggiunto `spring-boot-starter-validation`).

137. Crea un endpoint GET `/api/studenti/{id}/media` che calcola e restituisce la media voti dello studente come JSON: `{"studente": "Mario Rossi", "media": 27.5, "totaleEsami": 4}`.
138. Implementa la cancellazione a cascata: elimina uno studente e verifica che i suoi esami vengano eliminati automaticamente.

 **Consegna:** L'API deve gestire correttamente il caso in cui si tenti di aggiungere un esame con voto non valido (es. 15): deve rispondere con 400 Bad Request e un messaggio di errore esplicativo.

Giorno 17 — Advanced mappings: completamento e Many-to-Many

Esercizio 1 — Operazioni avanzate su relazioni One-to-Many

In questo esercizio consoliderai le operazioni di update e delete sulle entità con relazioni, gestendo correttamente le cascate e la pulizia dei dati orfani.

Cosa fare:

139. Partendo dal progetto esistente (Studente, CorsoLaurea, Esami), implementa l'aggiornamento di un corso: endpoint PUT /api/corsi/{id} che aggiorna nome e dipartimento del corso. Verifica che gli studenti associati rimangano invariati.
140. Implementa lo spostamento di uno studente da un corso a un altro: endpoint PATCH /api/studenti/{id}/corso con body {"corsold": 2}. Aggiorna la foreign key e verifica la coerenza dei dati.
141. Implementa la cancellazione di un corso. Cosa succede agli studenti? Prima del delete, decidi la strategia: (a) lancia un'eccezione se il corso ha studenti, oppure (b) imposta il corsoLaurea degli studenti a null. Implementa l'opzione (a).
142. Aggiungi @OrphanRemoval=true alla relazione Studente-Esami. Poi crea un endpoint DELETE /api/studenti/{id}/esami/{esameld} che rimuove un esame specifico dallo studente. Verifica che l'esame venga eliminato dal database.
143. Implementa un endpoint PUT /api/studenti/{id}/esami/{esameld} che aggiorna un esame (es. corregge il voto dopo ricorso). Carica lo studente, trova l'esame nella lista, aggiorna il voto, salva.
144. BONUS: Aggiungi una query JPQL che trovi tutti gli studenti con media superiore a un valore passato come parametro: findStudentiConMediaSuperioreA(double media). Crea nel repository con @Query.

 **Consegna:** Testa gli scenari edge case: cosa risponde l'API se si tenta di eliminare un corso con studenti? Cosa risponde se si aggiorna un esame con voto=17? Documenta i comportamenti nel README.

Esercizio 2 — Relazione Many-to-Many: Studenti e Attività

Modella la relazione Many-to-Many tra Studenti e Attività Extracurricolari: uno studente può partecipare a molte attività, e ogni attività può avere molti studenti.

Cosa fare:

145. Crea un'entità AttivitaExtracurricolare con: id, nome (String), tipo (String: 'Sport', 'Arte', 'Informatica', ...), crediti (int).
146. In Studente aggiungi @ManyToMany @JoinTable(name="studente_attivita", joinColumns=@JoinColumn(name="studente_id"), inverseJoinColumns=@JoinColumn(name="attivita_id")) List<AttivitaExtracurricolare> attivita.
147. In AttivitaExtracurricolare aggiungi @ManyToMany(mappedBy="attivita") List<Studente> partecipanti.
148. Nel DataLoader, crea 4 attività e iscrivine 2-3 studenti a ciascuna (con sovrapposizioni: lo stesso studente può partecipare a più attività). Verifica nella H2 Console la tabella di join STUDENTE_ATTIVITA.
149. Crea i seguenti endpoint: - GET /api/attivita → lista tutte le attività - GET /api/attivita/{id}/studenti → studenti iscritti a un'attività - POST

/api/studenti/{id}/attivit /{attivit id} → iscrive uno studente a un'attivit  - DELETE
/api/studenti/{id}/attivit /{attivit id} → rimuove uno studente da un'attivit 

150. Implementa la query: GET /api/studenti/{id}/crediti che somma i crediti di tutte le attiv   a cui partecipa lo studente e restituisce il totale.

151. Gestisci il caso in cui si tenti di iscrivere uno studente a un'attivit  a cui   gi  iscritto: controlla prima dell'inserimento e rispondi con 409 Conflict e un messaggio appropriato.

 **Consegna:** Testa l'intera relazione Many-to-Many con Postman. Poi scrivi un commento nel codice che spieghi perch  la tabella di join (STUDENTE_ATTIVITA) non   un'entit  JPA separata in questo caso, e quando invece converrebbe farne una.

Giorno 18 — Spiegazione del progetto finale

Esercizio 1 — Progettazione dell'architettura

Ora che hai ricevuto la spiegazione del progetto finale, è il momento di pianificarlo concretamente. Questo esercizio è di progettazione: non scriverai ancora codice, ma definirai la struttura su carta (o in un documento).

Cosa fare:

152. Scegli il dominio del tuo progetto (se non è stato assegnato). Descrivi in 5-10 righe cosa fa l'applicazione, chi sono gli utenti e quali sono le funzionalità principali.
153. Identifica le entità del dominio e le relazioni tra loro. Disegna un diagramma ER con: nomi delle tabelle, colonne principali, tipo di relazione (1:1, 1:N, N:M). Usa carta o uno strumento come draw.io.
154. Definisci gli endpoint REST dell'applicazione. Per ogni risorsa principale, elenca gli endpoint CRUD con: metodo HTTP, URL, descrizione, chi può accedervi (ruolo).
155. Pianifica i livelli dell'applicazione back-end: elenca i package che creerai (entity, repository, service, controller, security, config) e scrivi per ogni package quali classi conterrà.
156. Descrivi il front-end: quali pagine ci saranno? Come comunicherà con il back-end (chiamate AJAX/fetch)? Fai uno schizzo delle schermate principali.
157. Identifica i rischi e le difficoltà: qual è la parte che ti sembra più complessa? Quale argomento studiato questa settimana ti sembra meno chiaro? Questi sono i punti su cui chiedere aiuto prima di iniziare a sviluppare.

 **Consegna:** Consegna un documento (anche scritto a mano e fotografato) con: descrizione del progetto, diagramma ER, lista degli endpoint, struttura dei package, schizzi del front-end. Questo documento servirà come guida durante lo sviluppo.

Esercizio 2 — Setup del progetto e primo commit

Inizia concretamente il progetto: configura il repository Git, crea il progetto Spring Boot con le dipendenze giuste e imposta la struttura base. L'obiettivo è partire con un progetto funzionante, anche se ancora vuoto.

Cosa fare:

158. Crea un nuovo repository Git (locale, o su GitHub/GitLab se hai un account). Inizializza con un README.md che descrive il progetto (cosa fa, tecnologie usate).
159. Genera il progetto Spring Boot su start.spring.io con TUTTE le dipendenze che ti serviranno: Spring Web, Spring Data JPA, Spring Security, il database scelto (H2 per sviluppo o MySQL), Validation, SpringDoc OpenAPI. Importalo nell'IDE.
160. Configura application.properties per l'ambiente di sviluppo: datasource, show-sql=true, ddl-auto=create-drop (per ora), h2-console abilitata.
161. Crea la struttura dei package vuota: entity, repository, service, controller, security, config. Aggiungi in ogni package un file .gitkeep o una classe placeholder vuota.
162. Crea almeno la prima entità (quella centrale del tuo dominio) con i campi base e annota con @Entity. Crea il relativo Repository. Avvia l'applicazione e verifica che la tabella venga creata in H2.
163. Fai il primo commit con tutti questi file. Il messaggio del commit deve essere descrittivo: 'Initial project setup: Spring Boot configuration + first entity [NomeEntità]'. Poi crea un branch develop dal main per lo sviluppo.

 **Consegna:** Il progetto deve compilare e avviarsi senza errori. Il repository deve avere almeno 2 branch (main e develop) e almeno 1 commit. Condividi il link al repository (se pubblico) o mostra la struttura del progetto al docente.